**Course Name:** PCCCS495 – Object Oriented Programming Lab
**Project Title:** Hospital Appointment Booking System (Java Swing)
**Student Name:** Sampurna Rakshit
**Roll Number:** *12024052004068*
**Instructor Name:** *Dr. Soumyadip Biswas , Dr. Susovan Jana*
**Semester:** Spring 2026

# 2. Problem Statement

Manual hospital appointment systems are inefficient and error-prone. Patients often face issues such as long waiting times, difficulty in scheduling appointments, and lack of proper record management. Traditional systems rely heavily on manual processes, which can lead to data loss and mismanagement.

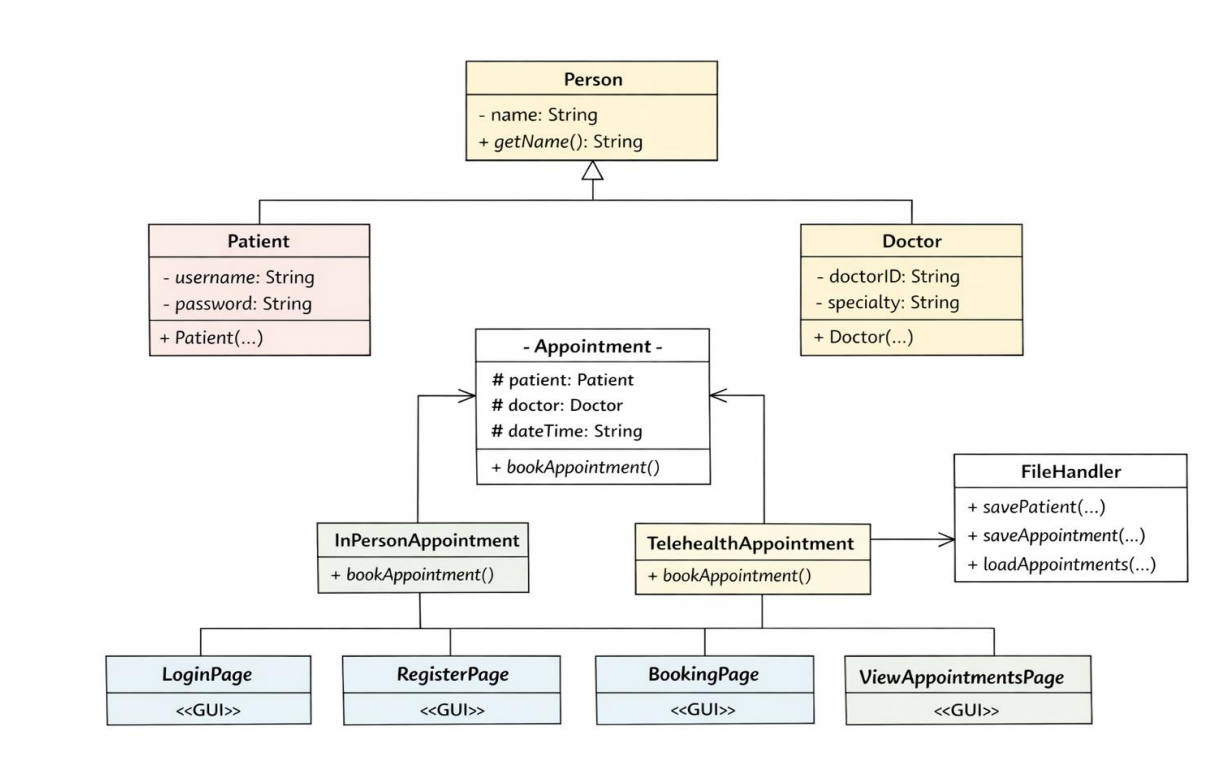
The objective of this project is to develop a simple Hospital Appointment Booking System using Java Swing GUI. The system allows users to register, log in, book appointments with doctors, and view previously booked appointments. It ensures better organization of data using file handling and improves usability through a graphical interface.

This project demonstrates the practical application of object-oriented programming concepts and provides a functional solution for small-scale appointment management.

---

# 3. System Design

## 3.1 UML Class Diagram (Description)



Classes:

- Person (Base class)
- Patient (inherits Person)
- Doctor (inherits Person)
- Appointment (Abstract class)
- InPersonAppointment (extends Appointment)
- TelehealthAppointment (extends Appointment)
- FileHandler (utility class)
- GUI Classes (LoginPage, RegisterPage, BookingPage, ViewAppointmentsPage)

---

### 3.2 Description of Major Classes

- **Patient:** Stores user details like name, username, password
- **Doctor:** Stores doctor details (name, specialization)
- **Appointment:** Abstract class representing appointment
- **InPersonAppointment / TelehealthAppointment:** Specific types
- **FileHandler:** Handles file read/write
- **GUI Classes:** Manage user interaction

---

### 3.3 Package Structure

```
src/
├── Main.java
├── GUI Classes
├── Model Classes
├── FileHandler.java
```

---

### 3.4 Interaction Flow

1. User opens application
2. Registers or logs in
3. System validates credentials
4. User books appointment
5. Data stored in file
6. User can view appointments

---

## 4. OOP Concepts Demonstrated

---

## 4.1 Abstraction

**Where used:** Appointment class

**Code Snippet:**

```
public abstract class Appointment {
    protected Patient patient;
    protected Doctor doctor;
    protected String dateTime;

    public abstract void bookAppointment();
}
```

**Explanation:**
The abstract class hides implementation details and provides a base structure for different appointment types.

---

## 4.2 Inheritance

**Where used:** Patient and Doctor extend Person

```
public class Patient extends Person {
    private String username;
    private String password;
}
```

**Explanation:**
Inheritance promotes code reuse and establishes relationship between classes.

---

## 4.3 Polymorphism

**Where used:** Appointment reference

```
Appointment appointment;

if (type.equals("In-Person")) {
    appointment = new InPersonAppointment(...);
} else {
    appointment = new TelehealthAppointment(...);
}
```

**Explanation:**
Same reference behaves differently based on object type.

---

### 4.4 Exception Handling

**Where used:** Input validation

```
if (date.isEmpty()) {
    throw new Exception("Date cannot be empty");
}
```

**Explanation:**
Prevents invalid input and ensures smooth execution.

---

### 4.5 Collections

**Where used:** ArrayList

```
ArrayList<Appointment> appointments = new ArrayList<>();
```

**Explanation:**
Used to store multiple appointments dynamically.

---

## 5. Implementation Highlights

---

### Snippet 1: Login Validation

```
ArrayList<Patient> patients = FileHandler.loadPatients();

for (Patient p : patients) {
    if (p.getUsername().equals(username) &&
        p.getPassword().equals(password)) {
        found = true;
    }
}
```

**Justification:**
Ensures only valid users can access the system.

---

### Snippet 2: File Saving

```
BufferedWriter writer = new BufferedWriter(
    new FileWriter("patients.txt", true));
writer.write(data);
```

**Justification:**
Stores user data permanently.

---

### Snippet 3: Appointment Booking

```
appointments.add(appointment);
FileHandler.saveAppointment(appointment);
```

**Justification:**
Combines in-memory storage and file storage.

---

## 6. Testing & Error Handling

---

### Test Cases

- Valid login credentials
- Invalid login
- Empty fields
- Booking with valid input

---

### Edge Cases

- Empty file
- Missing data
- Duplicate users

---

### Failure Scenarios

- File not found
- Incorrect input
- Corrupted data

---

## 7. Git Workflow

### Commit Strategy

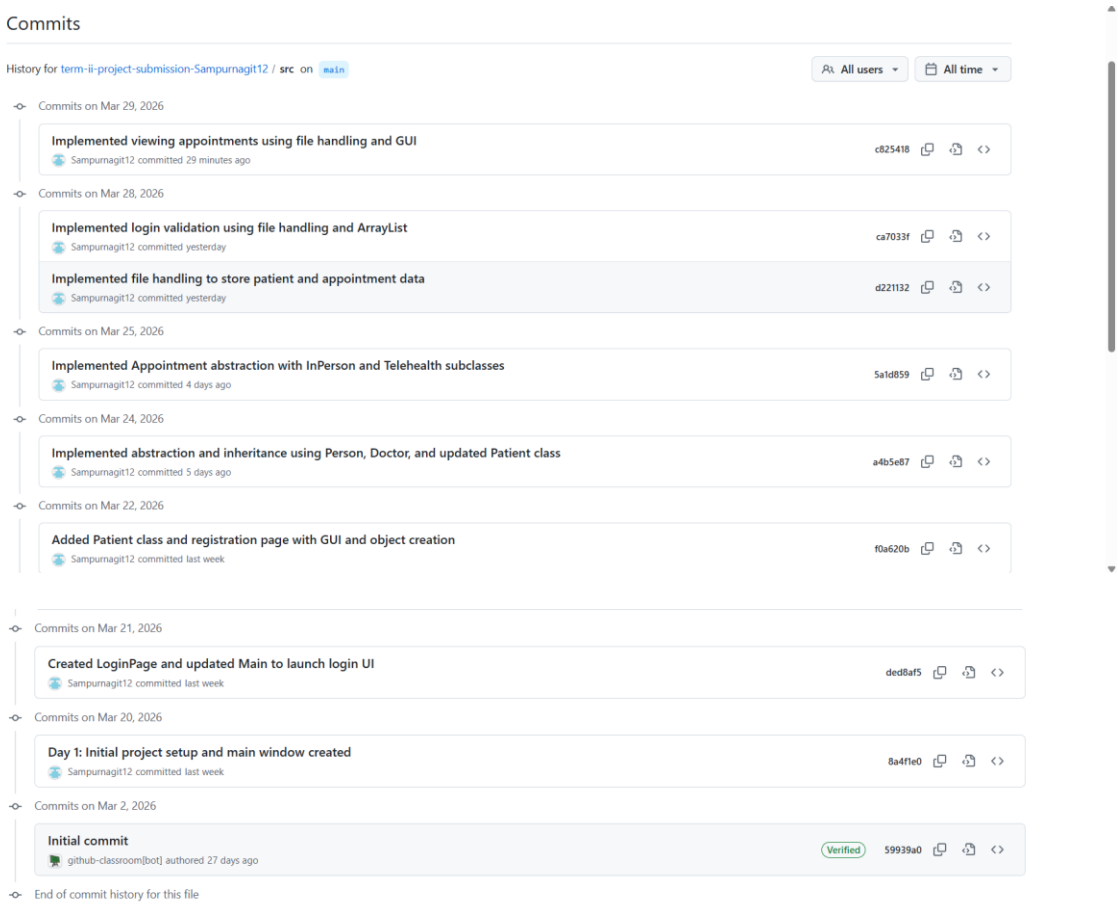
- Day 1: Setup

- Day 2–5: GUI + OOP
- Day 6–9: Features
- Day 10: Final polish

---

### Explanation

Each day's work was committed separately to maintain version control and track progress. GitHub was used to manage the repository and ensure structured development.




---

# 8. Conclusion & Future Scope

---

### Conclusion

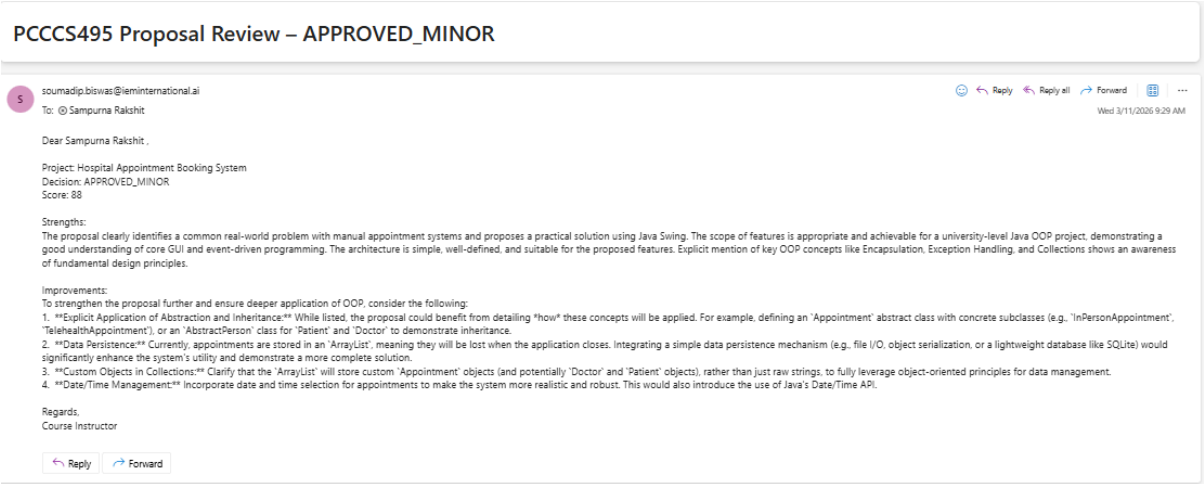
The project successfully implements a hospital appointment booking system using Java Swing. It demonstrates OOP principles, file handling, and GUI design. The system is functional, user-friendly, and meets the project requirements.

## Future Scope

- Database integration (Using ArrayList and File Handling)
- Improved UI design
- Appointment cancellation
- Admin panel

# Appendix

- Approval Mail (APPROVED_MINOR)



PCCCS495 Proposal Review – APPROVED_MINOR

soumadip.biswas@ieminternational.ai
To: Sampurna Rakshit

Reply Reply all Forward
Wed 3/11/2026 9:29 AM

Dear Sampurna Rakshit ,

Project: Hospital Appointment Booking System
Decision: APPROVED_MINOR
Score: 88

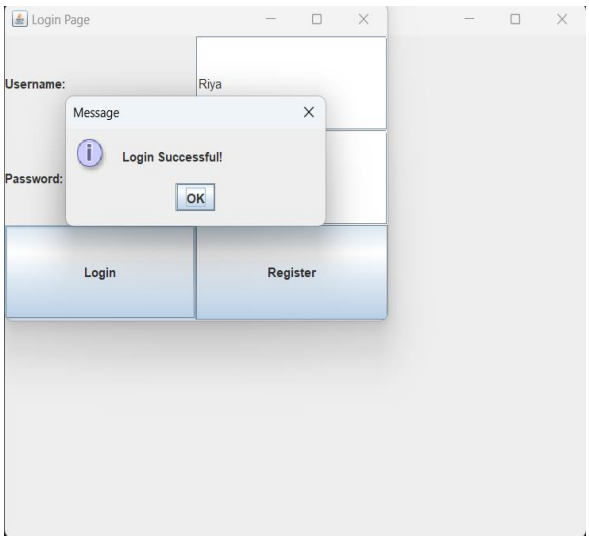
Strengths:
The proposal clearly identifies a common real-world problem with manual appointment systems and proposes a practical solution using Java Swing. The scope of features is appropriate and achievable for a university-level Java OOP project, demonstrating a good understanding of core GUI and event-driven programming. The architecture is simple, well-defined, and suitable for the proposed features. Explicit mention of key OOP concepts like Encapsulation, Exception Handling, and Collections shows an awareness of fundamental design principles.

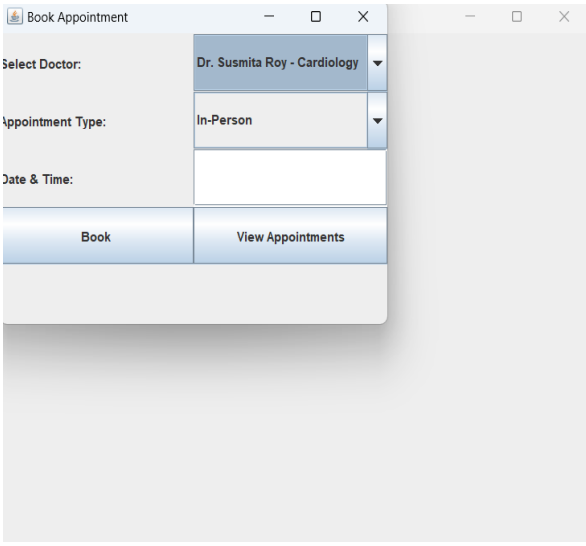
Improvements:
To strengthen the proposal further and ensure deeper application of OOP, consider the following:
1. **Explicit Application of Abstraction and Inheritance:** While listed, the proposal could benefit from detailing *how* these concepts will be applied. For example, defining an 'Appointment' abstract class with concrete subclasses (e.g., 'InPersonAppointment', 'TelehealthAppointment'), or an 'AbstractPerson' class for 'Patient' and 'Doctor' to demonstrate inheritance.
2. **Data Persistence:** Currently, appointments are stored in an 'ArrayList', meaning they will be lost when the application closes. Integrating a simple data persistence mechanism (e.g., file I/O, object serialization, or a lightweight database like SQLite) would significantly enhance the system's utility and demonstrate a more complete solution.
3. **Custom Objects in Collections:** Clarify that the 'ArrayList' will store custom 'Appointment' objects (and potentially 'Doctor' and 'Patient' objects), rather than just raw strings, to fully leverage object-oriented principles for data management.
4. **Date/Time Management:** Incorporate date and time selection for appointments to make the system more realistic and robust. This would also introduce the use of Java's Date/Time API.

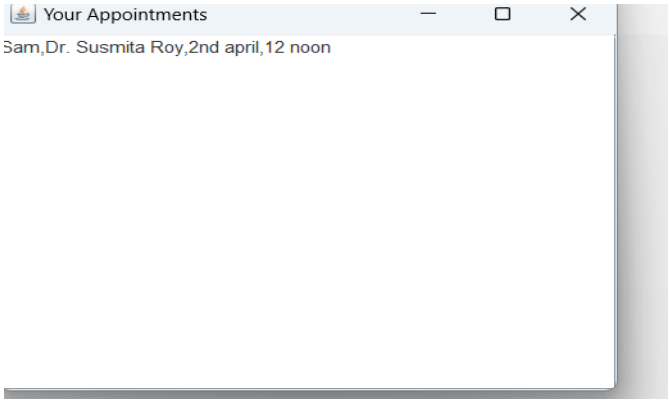
Regards,
Course Instructor

Reply Forward

- Screenshots of output






- GitHub repository link

  https://github.com/IEM-Gurukul/term-ii-project-submission-Sampurnagit12.git